

Week 1

1 Environment Setup

The notebook begins by preparing the Python environment required for working with astronomical images and DINOv2. The main libraries provide support for numerical computation, image processing, PyTorch-based deep learning, and visualization. The computation device is also identified so that inference can run on a GPU when available, otherwise falling back to the CPU.

2 Load DESI Dataset

The notebook loads galaxy imaging data from the DESI Legacy Survey. Each dataset entry contains astronomical image information along with associated metadata. Instead of loading the complete dataset into memory, the data can be accessed progressively, which is useful because astronomical datasets can contain a very large number of observations.

3 Dataset Exploration

The dataset is first inspected to understand its structure and the information available for each galaxy. This includes checking the number of samples, available fields, image representations, and relevant metadata. The purpose of this stage is to establish what information is available before sending the images through the vision model.

4 Visualize $g/r/i/z$

The DESI images contain four astronomical bands: g , r , i , and z . Each band represents the amount of detected flux within a particular wavelength range. Unlike a standard RGB photograph, these bands are scientific measurements of the observed light. Visualizing them separately helps verify the image data before it is converted into the RGB representation used by DINOv2.

5 Load Pretrained DINOv2

The notebook loads a pretrained DINOv2 Vision Transformer rather than training a model from scratch. DINOv2 learns visual representations using self-supervised learning, allowing the model to capture useful image features without requiring class labels for

every image. The model used here is DINOv2 ViT-S/14, where the patch size is 14×14 pixels.

6 Preprocess Image

Before an astronomical image can be passed to DINOv2, it must be converted into the format expected by the model. The RGB image is resized to 224×224 pixels, converted into a tensor, and normalized using the required input statistics. A batch dimension is also added so that a single image follows the standard PyTorch input format:

$$(1, 3, 224, 224).$$

Here, 1 represents the batch size and 3 represents the RGB channels.

7 Run Inference

The processed image is passed through DINOv2 in inference mode. No model parameters are updated during this step because the pretrained network is being used only to extract visual features. Gradient computation is disabled to reduce unnecessary memory usage and computation.

8 Extract CLS Token

The notebook extracts the CLS token from the internal representation of DINOv2. The CLS token provides a single vector that represents the image as a whole. For DINOv2 ViT-S/14, this representation has 384 dimensions:

$$\mathbf{z}_{CLS} \in R^{384}.$$

It can therefore be treated as a compact numerical representation of the entire galaxy image.

9 Extract Patch Tokens

In addition to the CLS token, the notebook extracts the representations corresponding to individual image patches. A 224×224 image with a patch size of 14×14 produces

$$\left(\frac{224}{14}\right)^2 = 16^2 = 256$$

patches. Each patch is represented by a 384-dimensional feature vector, giving a patch representation of approximately

$$256 \times 384.$$

These representations retain more local information than the single CLS vector.

10 Inspect Tensor Dimensions

The tensor dimensions are checked to verify that the extracted representations have the expected structure. This confirms the number of image patches, feature dimensions, and batch dimensions produced by the model. For a single 224×224 image, the expected patch representation contains 256 patch tokens, each with 384 features.

11 Visualize Patch Grid

The patch tokens correspond directly to the spatial regions created when the image is divided into 14×14 patches. Since there are 16 patches along each image dimension, the tokens can be arranged into a 16×16 grid. Visualizing this grid connects the numerical patch representations back to their original spatial locations in the galaxy image.

12 Initial Embedding Exploration

The final stage of the notebook begins exploring the extracted embeddings. The CLS embedding provides a representation of the complete galaxy, while the patch embeddings provide representations of individual regions. These vectors do not have simple human-readable meanings individually; their usefulness comes from comparing representations across different images. Similar visual structures can therefore be studied through distances or similarities between their embeddings.

For two embeddings $\mathbf{x}$ and $\mathbf{y}$, a common similarity measure is cosine similarity:

$$\text{cosine}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}.$$

This provides the foundation for later analysis such as comparing galaxies, clustering their representations, and identifying unusual objects.